# Unit 5 - Transaction Management

Database Management (DI03016041) - GTU Diploma IT Sem 3

**20% weightage | 9 hours**

**How to use these notes:** every topic is written in a descriptive, copy-ready way. **3 marks** = definition + 2-3 points. **4 marks** = add example or table. **7 marks** = write everything with diagram / example. Lines marked ★ show where the topic was asked in a GTU paper (Winter 2025 / Summer 2026). SQL examples use Oracle syntax.

## GTU questions already asked from this unit (do these first)

- Asked in BOTH papers: W2025 Q5(b)OR [4M: state transition diagram of transaction] + S2026 Q5(b)OR [4M: transaction states with diagram]
- ACID asked in BOTH papers: W2025 Q5(b) [4M: explain ACID properties] + S2026 Q5(c) [7M: list and discuss ACID properties]
- Asked in W2025 Q5(c) [7M: list problems of concurrency control, explain any two in detail]
- Locking and 2PL asked in BOTH papers: W2025 Q5(c)OR [7M: what is locking, explain 2PL with example] + S2026 Q5(a)OR [3M: explain 2PL technique]
- Asked in S2026 Q5(c)OR [7M: what is deadlock, detection and prevention]

## 5.1.1 Transaction definition and 5.1.4 transaction log

A **transaction** is a **logical unit of work** - a group of one or more database operations (read / write) that must be executed **completely or not at all**. It starts with the first SQL statement and ends with COMMIT or ROLLBACK.

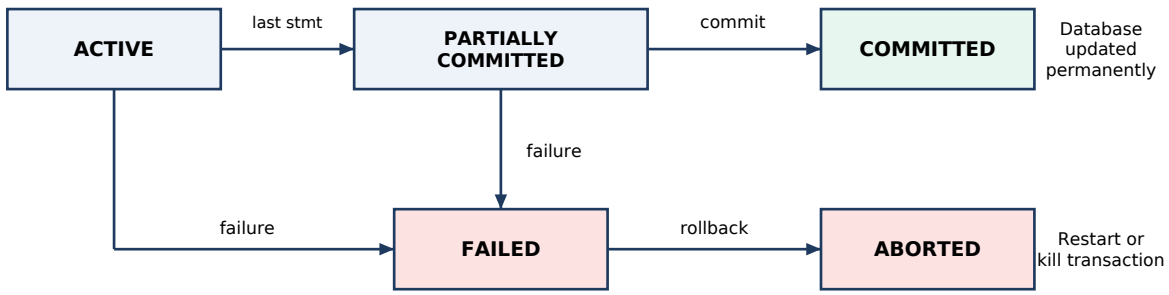
**Example:** Transferring Rs. 500 from account A to B: (1) READ A, (2) A = A - 500, (3) WRITE A, (4) READ B, (5) B = B + 500, (6) WRITE B. If the system fails after step 3, A has lost money but B has not gained - so either all six steps happen or none.

### Transaction log (5.1.4)

- A **transaction log (journal)** is a file in which the DBMS records **every change** made by transactions: transaction id, data item, **old value** and **new value**, and the start / commit / abort marks.
- **Uses: recovery** after a crash - committed transactions are **redone** (new values), incomplete ones are **undone** (old values); also rollback and audit.
- The log is written **before** the database is updated (write-ahead logging).

## 5.1.2 State transition diagram (transaction states)

**★ Asked in BOTH papers: W2025 Q5(b)OR [4M: state transition diagram of transaction] + S2026 Q5(b)OR [4M: transaction states with diagram]**




| State | Meaning |
|---|---|
| **Active** | Initial state; the transaction is executing its read / write operations |
| **Partially committed** | The final statement has been executed, but the changes are still in main memory (not yet permanently on disk) |
| **Committed** | The transaction completed successfully and its changes are **permanently stored** in the database |
| **Failed** | Normal execution can no longer continue (system error, logical error, power failure) |
| **Aborted** | The transaction has been **rolled back** and the database restored to its state before the transaction began. The DBMS may then **restart** the transaction or **kill** it. |

Path of a successful transaction: Active → Partially committed → Committed. Path of a failed one: Active (or Partially committed) → Failed → Aborted.

## 5.1.3 ACID properties

**★ ACID asked in BOTH papers: W2025 Q5(b) [4M: explain ACID properties] + S2026 Q5(c) [7M: list and discuss ACID properties]**

**ACID** properties are the four properties a transaction must have to keep the database **correct and reliable**:

| Property | Meaning | Example (transfer Rs. 500 from A to B; A = 2000, B = 1000) | Ensured by |
|---|---|---|---|
| **A - Atomicity** | **All or nothing**: either all operations of a transaction are done or none | Debit of A and credit of B both happen, or neither. If failure occurs after debit, the debit is rolled back | Transaction management (rollback, log) |
| **C - Consistency** | A transaction takes the database from **one valid (consistent) state to another**; all rules / constraints stay true | Before: A + B = 3000. After: A = 1500, B = 1500, total is still 3000 | Programmer, integrity constraints |
| **I - Isolation** | Concurrent transactions run **as if they were executed one after another**; intermediate results of one are invisible to others | Another transaction reading A and B in between does not see the money 'missing' | Concurrency control (locking) |
| **D - Durability** | Once committed, changes are **permanent** even after a crash or power failure | After commit message, A = 1500 and B = 1500 survive a system crash | Recovery system (log, backup) |

**Exam tip:** For 4 marks write the four properties with one meaningful line each. For 7 marks add the transfer example and 'ensured by'.

## 5.2.1-5.2.2 Concurrency control and its problems

**★ Asked in W2025 Q5(c) [7M: list problems of concurrency control, explain any two in detail]**

**Concurrency control** is the management of **simultaneous execution of transactions** so that they do not interfere with each other and the database stays consistent. Without it, these **problems** arise:

1. **Lost update problem**
2. **Dirty read (temporary update / uncommitted dependency) problem**
3. **Unrepeatable read problem**
4. **Incorrect summary (inconsistent analysis) problem**
5. **Phantom read problem**

### 1) Lost update problem (explained)

Two transactions read the same data item and update it; the update of one is **overwritten and lost**. Balance X = 100. T1 withdraws 10, T2 deposits 20. Correct final value = 110.

| Time | T1 | T2 | Value of X |
|---|---|---|---|
| t1 | READ(X) → 100 | | 100 |
| t2 | | READ(X) → 100 | 100 |
| t3 | X = X - 10 = 90 | | 100 |
| t4 | | X = X + 20 = 120 | 100 |
| t5 | WRITE(X) → 90 | | 90 |
| t6 | | WRITE(X) → 120 | **120** |

**Result:** X = 120 instead of 110; T1's update (withdrawal) is lost.

### 2) Dirty read problem (explained)

A transaction reads data written by another transaction that is **not yet committed**; if that transaction later **rolls back**, the first one has used invalid data. X = 100.

| Time | T1 | T2 |
|---|---|---|
| t1 | X = X + 50; WRITE(X) (X = 150, not committed) | |
| t2 | | READ(X) → 150 (dirty value) |
| t3 | ROLLBACK (X back to 100) | |
| t4 | | Continues using 150 - **wrong** |

### Others (one line each)

- **Unrepeatable read:** T1 reads X twice and gets different values because T2 changed X in between.
- **Incorrect summary:** T1 calculates a total (e.g. sum of balances) while T2 is updating some of those values, giving a wrong sum.
- **Phantom read:** T1 runs the same query twice and the second time sees extra rows inserted by T2.

## 5.2.3 Schedules and 5.3 serializability

### Schedule and its types (5.2.3)

A **schedule** is the **chronological order in which the operations of a set of transactions are executed**.

| Type | Meaning |
|---|---|
| **Serial schedule** | Transactions run **one after another**, no interleaving (T1 completely, then T2). Always consistent but slow (no concurrency). |
| **Non-serial (concurrent) schedule** | Operations of different transactions are **interleaved**. Faster, but may be inconsistent. |
| **Serializable schedule** | A non-serial schedule whose result is the **same as some serial schedule** - correct and concurrent. |
| **Recoverable schedule** | A transaction commits only after every transaction whose data it read has committed. |

### Serializability of transactions (5.3)

- A schedule is **serializable** if it is equivalent to a serial schedule. It is the main correctness criterion for concurrent execution.
- **Conflict serializability:** the schedule can be converted into a serial schedule by swapping **non-conflicting** operations. Two operations **conflict** if they belong to different transactions, access the **same data item**, and at least one is a **WRITE** (R-W, W-R, W-W).
- **Test:** draw a **precedence graph** (node = transaction; edge Ti → Tj if a conflicting operation of Ti comes before Tj). **No cycle ⇒ conflict serializable**; cycle ⇒ not serializable.
- **View serializability:** a weaker condition based on same initial reads, same final writes, and same read-from relationships.

## 5.4 Locking methods and two-phase locking

**★ Locking and 2PL asked in BOTH papers: W2025 Q5(c)OR [7M: what is locking, explain 2PL with example] + S2026 Q5(a)OR [3M: explain 2PL technique]**

**Locking** is a concurrency-control method in which a transaction must **obtain a lock on a data item before accessing it**, and other transactions have to wait while the item is locked. It guarantees isolation.

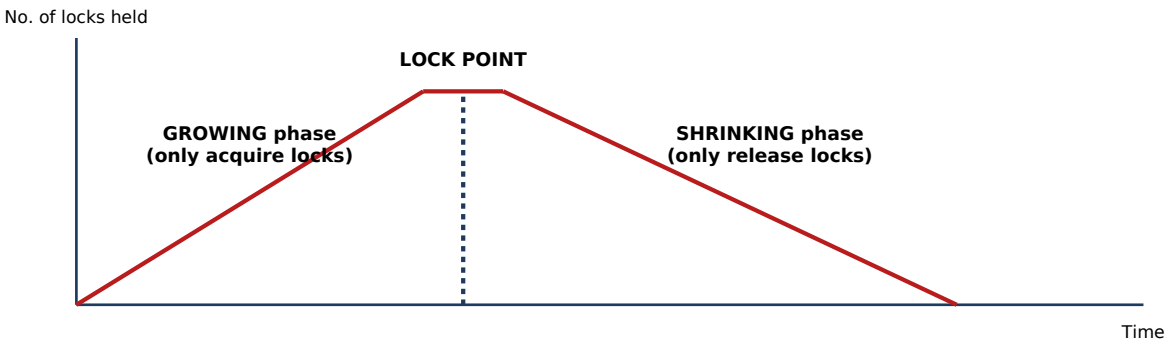
### Types of locks

| Lock | Purpose | Rule |
|---|---|---|
| **Shared lock (S) / read lock** | For reading a data item | Many transactions can hold S on the same item at once |
| **Exclusive lock (X) / write lock** | For reading and writing | Only one transaction can hold X; no other S or X allowed |

| | Held: S | Held: X |
|---|---|---|
| Request S | Granted | Wait |
| Request X | Wait | Wait |

### Two-Phase Locking (2PL) protocol

2PL requires every transaction to do its locking in **two phases**:



- **Growing (expanding) phase:** the transaction can **acquire (obtain) locks** but cannot release any lock.
- **Shrinking phase:** the transaction can **release locks** but cannot acquire any new lock.
- The moment the first lock is released is the end of the growing phase (**lock point**). Once it releases a lock, it can never lock again.

**2PL guarantees conflict-serializable schedules.**

### Example

| Step | Transaction T1 (transfer from A to B) | Phase |
|---|---|---|
| 1 | LOCK-X(A) | Growing |
| 2 | READ(A); A = A - 500; WRITE(A) | Growing |
| 3 | LOCK-X(B) | Growing (lock point) |
| 4 | READ(B); B = B + 500; WRITE(B) | - |
| 5 | UNLOCK(A) | Shrinking |
| 6 | UNLOCK(B) | Shrinking |

Another transaction T2 that wants A must wait until T1 unlocks A, so T1 and T2 cannot interfere.

**Variants: Strict 2PL** - all exclusive locks are held until the transaction commits or aborts (avoids cascading rollback). **Rigorous 2PL** - all locks held until commit.

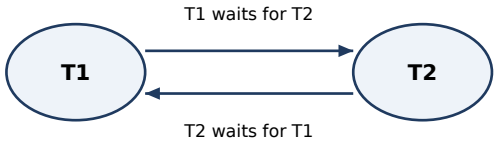
**Drawback:** 2PL does not prevent **deadlock** and can reduce concurrency.

## Deadlock: detection and prevention

**★ Asked in S2026 Q5(c)OR [7M: what is deadlock, detection and prevention]**

A **deadlock** is a situation in which **two or more transactions wait forever**, each waiting for a data item locked by another transaction in the set. None can proceed.

> **Example:** T1 holds a lock on A and requests B; T2 holds a lock on B and requests A. T1 waits for T2 and T2 waits for T1 - deadlock.




**Cycle in wait-for graph = DEADLOCK**

### Deadlock detection

- The DBMS builds a **wait-for graph**: a node for each transaction; an edge Ti → Tj when Ti is waiting for a lock held by Tj.
- The graph is checked periodically. **A cycle in the graph = deadlock.**
- **Recovery:** choose a **victim** transaction (usually the youngest / cheapest to roll back), **abort and roll it back** to release its locks, and restart it later. Avoid always picking the same victim (starvation).

### Deadlock prevention

- **Lock all items in advance (conservative 2PL):** a transaction locks every item it needs before it starts; if not all are available, it waits without holding any.
- **Ordering of resources:** all transactions must request locks in a fixed order, so no cycle can form.
- **Timestamp-based schemes** (older transaction has higher priority):
- **Wait-Die:** if an *older* transaction requests an item held by a younger one, it **waits**; if a *younger* requests an item held by an older one, it **dies** (rolls back and restarts).

- **Wound-Wait:** if an older transaction requests an item held by a younger one, it **wounds** (aborts) the younger; a younger one requesting from an older **waits**.
- **Timeout:** a transaction waiting longer than a set time is aborted.

| Point | Prevention | Detection |
|---|---|---|
| Idea | Make deadlock impossible | Allow it, find it, then break it |
| Cost | Unnecessary rollbacks, less concurrency | Overhead of checking graph and rollback of victim |
| Best when | Deadlocks are frequent | Deadlocks are rare |

# Last-minute revision sheet - Unit 5

- Transaction = logical unit of work; ends with COMMIT or ROLLBACK.
- States: Active → Partially committed → Committed; Active / Partially committed → Failed → Aborted.
- ACID: Atomicity (all or nothing), Consistency, Isolation, Durability.
- Concurrency problems: lost update, dirty read, unrepeatable read, incorrect summary, phantom.
- Serializable = equals some serial schedule; precedence graph with no cycle.
- 2PL: growing phase (only lock) then shrinking phase (only unlock); S and X locks.
- Deadlock = cyclic wait; detect with wait-for graph; prevent with wait-die / wound-wait.